Tecnologyc Store JACG

Documentación Técnica del Código

Alumno	Jose Alberto Garcia Cortes
Materia	TAP
Tema	Proyecto Integrador
Fecha	14/03/2026

Catálogo interactivo de productos tecnológicos

1. Introducción y Estructura General

Tecnologyc Store JAGC es una aplicación de escritorio desarrollada en Python usando el framework Flet. Muestra un catálogo visual de productos tecnológicos con tarjetas interactivas, efectos visuales al pasar el cursor, sistema de favoritos y carrito de compras funcional.

1.2 Estructura del código

El archivo techstore.py está dividido en 3 partes principales:

Parte	Nombre	Descripción
A	**products[]**	Lista con los datos de todos los productos (nombre, precio, imagen, etc.)
B	**ProductCard**	Componente visual reutilizable que dibuja cada tarjeta de producto
C	**main()**	Función principal que ensambla toda la interfaz y maneja el carrito

2. Parte A — Modelo de Datos (products[])

Esta sección define todos los productos que aparecen en el catálogo. Es una lista de diccionarios de Python, donde cada diccionario representa un producto.

2.1 Estructura de un producto

```
products = [
    {
        "id":          1,
        "nombre":      "MacBook Pro m4",
        "descripcion": "Laptop de alto rendimiento...",
        "precio":      19999.99,
        "categoria":   "Laptops",
        "ruta_imagen": "images.jpeg",
    },
    ...
]
```

Campo	Tipo	Descripción
id	int	Identificador único del producto
nombre	str	Nombre comercial que aparece en la tarjeta
descripcion	str	Texto descriptivo (máx. 3 líneas visibles)
precio	float	Precio en MXN, se formatea automáticamente con comas y 2 decimales
categoria	str	Categoría del producto. Determina el ícono y color del badge
ruta_imagen	str	Nombre del archivo de imagen dentro de la carpeta assets/

2.2 Diccionarios de categorías

Fuera de la lista de productos existen dos diccionarios adicionales que mapean cada categoría a un ícono y un color:

```
CATEGORY_ICONS  = { "Laptops": ft.Icons.LAPTOP_ROUNDED, ... }
CATEGORY_COLORS = { "Laptops": "#1565C0", ... }
```

Cuando se construye una tarjeta, el código busca en estos diccionarios la categoría del producto y obtiene automáticamente el ícono y color correctos para el badge.

💡 **¿Cómo agregar un producto nuevo?**

Solo agrega un nuevo diccionario al final de la lista products[] con los 6 campos. Asegúrate de que ruta_imagen coincida exactamente con el nombre del archivo en la carpeta assets/. Flet creará la tarjeta automáticamente.

3. ¿Cómo llegan las imágenes al código?

Esta es una de las partes más importantes para entender cómo funciona la aplicación visualmente.

3.1 El flujo completo de una imagen

La imagen viaja desde tu carpeta hasta la pantalla en 4 pasos:

- Guardas el archivo de imagen en la carpeta assets/ (ej: galaxy.jpeg)
- En el diccionario del producto escribes su nombre: "ruta_imagen": "galaxy.jpeg"
- ft.app(assets_dir="assets") le dice a Flet que sirva esa carpeta como servidor
- ft.Image(src="galaxy.jpeg") busca el archivo ahí y lo muestra en pantalla

```
assets/galaxy.jpeg
        ↓
ft.app(assets_dir="assets")
        ↓
ft.Image(src="galaxy.jpeg", fit=ft.BoxFit.CONTAIN)
        ↓
🖥️  Se muestra en la tarjeta
```

3.2 El parámetro assets_dir

Al final del código, la línea ft.app(target=main, assets_dir="assets") hace que Flet actúe como un mini servidor web que sirve automáticamente todos los archivos que estén dentro de la carpeta assets/. Tú solo indicas el nombre del archivo, y Flet hace el resto.

3.3 ft.BoxFit.CONTAIN

El parámetro fit=ft.BoxFit.CONTAIN le dice a Flet cómo escalar la imagen dentro del espacio disponible. CONTAIN significa que la imagen se ajusta completa sin cortarse, manteniendo su proporción original.

3.4 Imagen fallback (plan B)

Si una imagen no se encuentra o tiene error, el código muestra automáticamente un ícono de 'Sin imagen' en su lugar usando error_content:

```
ft.Image(
    src="galaxy.jpeg",
    error_content=ft.Container(
```

```
        content=ft.Icon(ft.Icons.IMAGE_NOT_SUPPORTED_OUTLINED),
    )
)
```

3.5 Estructura de carpetas requerida

```
mi_proyecto/
├── techstore.py
└── assets/
    ├── images.jpeg    ← MacBook
    ├── galaxy.jpeg    ← Samsung
    ├── sony.jpeg      ← Audífonos
    ├── ipad.jpeg      ← iPad
    └── monitor.jpeg   ← LG Monitor
```

> ⚠️ **Importante**
> El nombre del archivo en ruta_imagen debe ser EXACTAMENTE igual al nombre real del archivo, incluyendo la extensión (.jpeg, .png, .svg). Los sistemas de archivos en Linux/Mac distinguen mayúsculas de minúsculas.

4. Parte B — ProductCard (Tarjeta de Producto)

ProductCard es el corazón visual de la aplicación. Es una clase que hereda de ft.Container y sabe cómo dibujarse a sí misma a partir de un diccionario de producto.

4.1 ¿Qué significa heredar de ft.Container?

En Python, cuando una clase hereda de otra, obtiene todos sus atributos y métodos. Al heredar de ft.Container, ProductCard ES un contenedor de Flet con todas sus propiedades (bgcolor, shadow, border_radius, etc.) y además agrega su propia lógica de construcción.

```
class ProductCard(ft.Container):  # ← hereda TODO de ft.Container
    def __init__(self, product, on_add_to_cart=None):
        super().__init__()  # ← inicializa ft.Container primero
        self.product = product  # guarda los datos del producto
```

4.2 El constructor __init__

El constructor es la función que se ejecuta cuando se crea una tarjeta. Define todas las propiedades visuales del contenedor externo y al final llama a _build() para construir el contenido interno.

Propiedad	¿Qué hace?
self.width = 275	Ancho fijo de la tarjeta en píxeles
self.border_radius	Redondea las esquinas de la tarjeta (radio de 20px)
self.bgcolor	Color de fondo blanco
self.shadow	Sombra suave debajo de la tarjeta para efecto de elevación
self.animate	Hace que los cambios de estilo sean animados (180ms)
self.animate_scale	Permite animar el tamaño (escala) de la tarjeta suavemente
self.content = self._build()	Construye y asigna todo el contenido visual interno

4.3 La función _build() — El corazón visual

Esta función construye todo lo que se ve dentro de la tarjeta. Retorna una columna vertical (ft.Column) con 4 secciones apiladas:

Sección	Componente	Contiene
1	**image_area**	Imagen del producto + badge de categoría superpuesto
—— *divider* ——	ft.Divider	Línea separadora gris de 1px
2	**text_body**	Nombre, descripción, precio y chip de disponibilidad
—— *divider* ——	ft.Divider	Línea separadora gris de 1px
3	**action_bar**	Botón de favorito (corazón) y botón Al carrito

4.3.1 image_area — La imagen con badge

Usa ft.Stack, que apila controles uno encima del otro como capas:

- Capa inferior: ft.Image mostrando la foto del producto
- Capa superior: el badge de categoría en la esquina superior izquierda

```
ft.Stack(controls=[
    ft.Container(content=ft.Image(src='galaxy.jpeg')),  # ← capa de abajo
    ft.Container(content=badge, alignment=ft.Alignment(-1,-1)),  # ← esquina sup.izq
])
```

El badge se crea usando los diccionarios CATEGORY_ICONS y CATEGORY_COLORS. Para la categoría 'Smartphones' obtendrá el ícono de smartphone y el color morado.

4.3.2 text_body — El texto

Es un ft.Container con un ft.Column adentro que muestra el nombre en negrita, la descripción en gris, y el precio en verde. También incluye un pequeño chip verde de 'Disponible' a la derecha del precio.

```
ft.Text(self.product['nombre'], weight=ft.FontWeight.BOLD, size=15)
ft.Text(self.product['descripcion'], max_lines=3)  # máx. 3 líneas
ft.Text(f"$ {self.product['precio']:,.2f} MXN")  # formatea el número
```

4.3.3 action_bar — Los botones

Contiene dos controles en fila horizontal (ft.Row):

- ft.IconButton: el corazón de favorito. Tiene estado propio (_favorito) y se actualiza solo
- ft.Button: el botón 'Al carrito'. Cuando se presiona, llama a on_add_to_cart

4.4 La función _on_hover() — Efecto al pasar el mouse

Flet llama a esta función automáticamente cada vez que el mouse entra o sale de la tarjeta. El parámetro e.data es el string 'true' cuando el mouse entra y 'false' cuando sale.

```
def _on_hover(self, e):
    hovered = str(e.data).lower() == 'true'  # ¿está el mouse encima?

    # Cambia la sombra según el estado
    self.shadow = self._shadow_hover() if hovered else self._shadow_normal()

    # Escala la tarjeta 3% más grande al hacer hover
    self.scale = ft.Scale(scale=1.03 if hovered else 1.0)

    self.update()  # ← redibuja la tarjeta con los cambios
```

4.5 La función _toggle_fav() — El favorito

Alterna el estado del favorito cada vez que el usuario hace clic en el corazón. Usa self._favorito como variable de estado (True = favorito, False = no favorito).

```
def _toggle_fav(self, e, btn):
    self._favorito = not self._favorito  # invierte: True→False, False→True

    # Cambia el ícono según el nuevo estado
    btn.icon = ft.Icons.FAVORITE_ROUNDED if self._favorito else
ft.Icons.FAVORITE_BORDER_ROUNDED
    btn.icon_color = ft.Colors.RED_500 if self._favorito else ft.Colors.RED_300

    btn.update()  # ← redibuja solo el botón (más eficiente)
```

5. Parte C — Función main() (Página Principal)

La función main() es el punto de ensamblaje de toda la aplicación. Recibe el objeto page (que representa la ventana completa) y agrega todos los componentes visuales en orden de arriba hacia abajo.

5.1 Configuración de la página

```
def main(page: ft.Page):
    page.title   = "TechStore"         # título de la ventana
    page.bgcolor = "#EEF2F7"           # color de fondo azul-gris claro
    page.padding = 0                   # sin margen interno
    page.scroll  = ft.ScrollMode.AUTO  # scroll si el contenido no cabe
    page.theme   = ft.Theme(font_family="Poppins")  # fuente global
```

5.2 El sistema del carrito

El carrito es una lista vacía que crece cada vez que el usuario agrega un producto. La función add_to_cart() es la pieza clave:

```
cart: list[dict] = []  # lista vacía al inicio


def add_to_cart(product: dict):
    cart.append(product)               # 1. guarda el producto en la lista
    cart_badge.visible = True          # 2. muestra el badge rojo
    cart_count.value = str(len(cart))  # 3. actualiza el número del badge

    snack = ft.SnackBar(...)           # 4. crea la notificación
    page.overlay.append(snack)         # 5. la agrega a la pantalla
    snack.open = True                  # 6. la muestra
    page.update()                      # 7. redibuja toda la página
```

5.3 La conexión entre ProductCard y add_to_cart

Cuando se crea cada tarjeta, se le pasa la función add_to_cart como parámetro. Esto funciona como darle un número de teléfono a la tarjeta: cuando el botón se presiona, la tarjeta llama a esa función.

```
# En main(), al crear cada tarjeta:
ProductCard(product=p, on_add_to_cart=add_to_cart)

# Dentro de ProductCard, el botón usa esa función:
ft.Button(
    on_click=lambda e, p=self.product: self.on_add_to_cart(p)
```

```
)
```

🔗 **Concepto clave: Callbacks**

Pasar una función como parámetro (on_add_to_cart=add_to_cart) se llama callback. ProductCard no sabe qué hace add_to_cart, solo la llama cuando el botón se presiona. Esto hace el código modular y reutilizable.

5.4 Componentes de la interfaz

La página se construye con 5 bloques que se apilan verticalmente:

Componente	Variable	Descripción
1. Header	header	Barra superior con logo TechStore, gradiente azul y botón del carrito con badge contador
2. Stats Bar	stats_bar	Fila de 4 chips blancos con estadísticas: cantidad de productos, envío gratis, garantía y pagos seguros
3. Título	section_title	Barra azul vertical decorativa + texto 'Artículos Destacados' con subtítulo de conteo
4. Tarjetas	cards_grid	ft.Row con wrap=True que crea una tarjeta ProductCard por cada producto. Las tarjetas bajan a la siguiente fila si no caben
5. Footer	footer	Pie de página con links de Política, Términos y Contacto, más el texto de derechos reservados

5.5 El grid de tarjetas — ft.Row con wrap

El grid que muestra todas las tarjetas usa una Row (fila) con wrap=True, lo que significa que si las tarjetas no caben en una sola fila, automáticamente bajan a la siguiente:

```
ft.Row(
    wrap=True,        # ← baja a la siguiente fila si no caben
    spacing=22,       # espacio horizontal entre tarjetas
    run_spacing=22,   # espacio vertical entre filas
    alignment=ft.MainAxisAlignment.CENTER,  # centrado
    controls=[
        ProductCard(product=p, on_add_to_cart=add_to_cart)
        for p in products  # ← crea UNA tarjeta por cada producto
    ],
)
```

5.6 Ensamblaje final

Al final de main(), todos los componentes se agregan a la página en orden. page.add() los apila de arriba hacia abajo:

```
page.add(header, stats_bar, section_title, cards_grid, footer)
```

6. Flujo Completo de la Aplicación

Este diagrama muestra cómo interactúan todas las partes cuando el usuario usa la aplicación:

```
1. ft.app() arranca la aplicación
        ↓
2. main(page) configura la ventana y el carrito
        ↓
3. Para cada producto en products[]:
       ProductCard(product=p, on_add_to_cart=add_to_cart)
           → __init__() configura estilos
           → _build() dibuja imagen + texto + botones
        ↓
4. page.add(header, stats, title, grid, footer)
        ↓
--- EN USO ---
Mouse entra → _on_hover('true') → sombra+escala → update()
Clic ❤️     → _toggle_fav()     → cambia ícono  → btn.update()
Clic 🛒     → on_add_to_cart()  → cart.append() → page.update()
```

6.1 Resumen de funciones por responsabilidad

Función	Ubicación	Responsabilidad
__init__()	ProductCard	Configura estilos y llama a _build()
_build()	ProductCard	Construye imagen, texto y botones. Retorna ft.Column
_shadow_normal()	ProductCard	Retorna la sombra suave por defecto
_shadow_hover()	ProductCard	Retorna la sombra más intensa y azul al hacer hover
_on_hover(e)	ProductCard	Detecta si el mouse está encima y actualiza sombra y escala
_toggle_fav(e, btn)	ProductCard	Alterna el estado del favorito y actualiza el ícono del botón
main(page)	Global	Configura la página, crea componentes y los ensambla
add_to_cart(product)	Dentro de main()	Agrega producto al carrito, actualiza badge y muestra snackbar
stat_chip(icon, label, value)	Dentro de main()	Helper que crea cada chip de estadística (reutilizable)